SISTEMAS DISTRIBUÍDOS

Implementando Flask utilizando **Banco de Dados**

(Flask + SQLite)

FLASK + SQLite

Propósito: substituir o dicionário em memória por uma **tabela no banco**.

Para isso teremos:

- `servidor.py` usando Flask + SQLite
- `cliente.py` usando requests
- criação automática do banco
- rota para listar produtos
- rota para compras
- rota para listar pedidos

FLASK + SQLite em um sistema de loja online

Exemplo: servidor Flask com comunicação com um banco de dados para armazenar e consultar informações de uma loja online.

- clientes enviam requisições HTTP e o servidor registra os dados no banco SQLite.

FLASK + SQLite em um sistema de loja online

Exemplo: servidor Flask com comunicação com um banco de dados para armazenar e consultar informações de uma loja online.

- Banco de dados com duas entidades principais:
  - produto e
  - pedido dos clientes.

FLASK + SQLite em um sistema de loja online

Exemplo: servidor Flask com comunicação com um banco de dados para armazenar e consultar informações de uma loja online.

- Banco de dados com duas entidades principais:
  - produto e
  - pedido dos clientes.

FLASK

Serviço

SQLite

Persistência dos dados

disciplinas e as matrículas permanecem salvas, permitindo o funcionamento do sistema mesmo após reinicializações.

FLASK + SQLite em um sistema de loja online

SQLite: Biblioteca C que fornece um banco de dados que não requer um processo de servidor separado e permite acessar o banco de dados usando linguagem de consulta SQL.

Para criar um novo banco de dados e abrir uma conexão de banco de dados:

```
import sqlite3
sqlite3.connect()
```

Exemplo: criando uma conexão com base de dados teste.db

```
conn = sqlite3.connect("app_loja.db")
```

Caso o banco não exista, ele é criado no diretório de trabalho atual.

FLASK + SQLite em um sistema de loja online

SQLite: Biblioteca C que fornece um banco de dados que não requer um processo de servidor separado e permite acessar o banco de dados usando linguagem de consulta SQL.

Para a busca de resultados e execução de instruções SQL, precisaremos usar um cursor de banco de dados:

```
cursor = conn.cursor() # cria o cursor
```

FLASK + SQLite em um sistema de loja online

SQLite: Biblioteca C que fornece um banco de dados que não requer um processo de servidor separado e permite acessar o banco de dados usando linguagem de consulta SQL.

Criando tabelas:

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS produto (
        codigo TEXT PRIMARY KEY,
        nome TEXT NOT NULL
    )
""")
```

FLASK + SQLite em um sistema de loja online

SQLite: Biblioteca C que fornece um banco de dados que não requer um processo de servidor separado e permite acessar o banco de dados usando linguagem de consulta SQL.

Criando tabelas - comando INSERT IGNORE: usado para inserir registros no banco sem duplicar registros já existentes.

```
cursor.execute("""
    INSERT OR IGNORE INTO produto (codigo, nome)
    VALUES
    ('CP000001', 'camisa'),
    ('CP000002', 'calça'),
    ('CP000003', 'vestido')
""")
```

FLASK + SQLite em um sistema de loja online

SQLite: Biblioteca C que fornece um banco de dados que não requer um processo de servidor separado e permite acessar o banco de dados usando linguagem de consulta SQL.

Integridade dos dados: antes de inserir dados no banco, o servidor realiza validações:

- verifica, por exemplo, se o JSON foi enviado
- se os campos obrigatórios estão preenchidos
- se o produto existe,
- se ainda há produtos no estoque disponíveis

Essas validações são fundamentais para evitar inconsistências e manter o banco em um estado correto.

FLASK + SQLite em um sistema de loja online

SQLite: Biblioteca C que fornece um banco de dados que não requer um processo de servidor separado e permite acessar o banco de dados usando linguagem de consulta SQL.

Para confirmar alterações no banco de dados:

```
conn.commit()
```

Para fechar a conexão com o banco de dados:

```
conn.close()
```

A instrução libera recursos e evita problemas causados por conexões abertas desnecessariamente.

FLASK + SQLite em um sistema de loja online

SQLite: Biblioteca C que fornece um banco de dados que não requer um processo de servidor separado e permite acessar o banco de dados usando linguagem de consulta SQL.

Consulta de estoque (`GET /estoque`): rota que permite ao cliente consultar os produtos cadastrados no banco de dados.

- O servidor abre a conexão, executa um comando SELECT na tabela produto, coleta os resultados e os organiza em uma lista no formato JSON.

```
...
    cursor.execute(
        "SELECT codigo, nome, quantidade FROM produto"
    )
    info = cursor.fetchall()
    conn.close()
...
```

FLASK + SQLite em um sistema de loja online

Rota de compra (`POST /comprar`): rota que recebe do cliente o nome do produto e a quantidade desejada.

- O servidor valida os dados, consulta o banco para verificar se o item existe e se há estoque disponível. Após, o sistema atualiza a quantidade do produto, registra o pedido e devolve uma resposta em JSON com o resultado da operação.

```
...
    cursor.execute(
        "SELECT nome, quantidade FROM produto WHERE nome = ?",(produto,)
    )
    item = cursor.fetchone()
...
    cursor.execute(
        "UPDATE produto SET quantidade = quantidade - ? WHERE nome = ?",(qtd,produto)
    )
    cursor.execute(
        "INSERT INTO pedido (nome_produto,quantidade_produto) VALUES (?, ?)",
    (produto, qtd)
    )
...
```